
**University of Dhaka**

Department of Computer Science and Engineering

## CSE 3111: Computer Networking Lab

---

# Lab Report - 2

---

## Implementing File Transfer using Socket Programming and HTTP GET/POST requests

**Lab Group: A (ODD)**

**H.M. Mehedi Hasan** (Roll: 13)
**MD. Abu Bakar Siddique** (Roll: 47)

**Submission Date: September 24, 2025**

# Contents

# 1 Introduction

File transfer is a crucial feature of networking applications, allowing files to be shared between clients and servers over a network. Traditionally, raw socket programming is used, but modern applications often prefer HTTP protocols.

HTTP provides a higher-level abstraction where:

- **GET** requests are used for downloading files.
- **POST** requests are used for uploading files.

# 2 Objectives

1. To implement a file transfer system using HTTP GET (download) and POST (upload) methods in Java.
2. To demonstrate practical differences between raw socket programming and HTTP protocol-based communication.
3. To analyze the advantages of HTTP-based file transfer in real-world applications.

# 3 Design Details

## 3.1 Implementation Process

1. **Server:**
   (a) Initialize an HTTP server on port `8080`.
   (b) Configure `/upload` endpoint to handle file uploads with POST.
   (c) Configure `/download` endpoint to send files via GET.
   (d) Save uploaded files inside the `uploads/` directory.
2. **Client:**
   (a) Present user menu: upload or download.
   (b) Upload: send file using HTTP POST to `/upload`.
   (c) Download: request file using HTTP GET from `/download?filename={name}`.
   (d) Save received files inside the `download/` directory.

# 4 Implementation

## 4.1 Server Flowchart

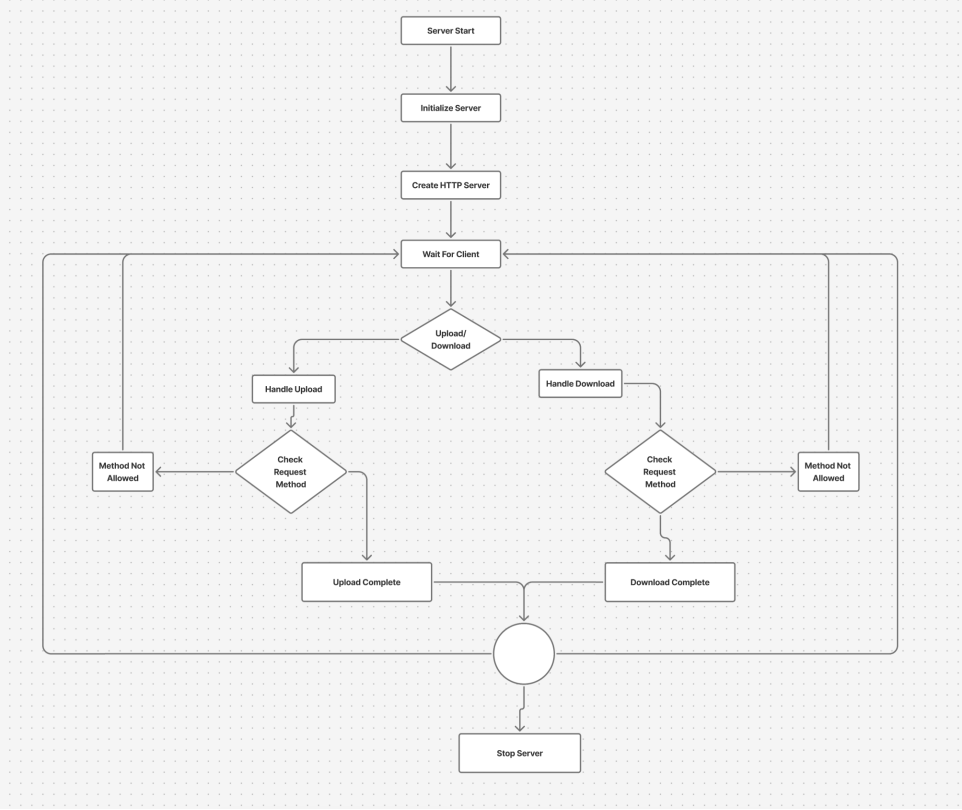


Figure 1: Flowchart of the File Server

## 4.2 Client Flowchart

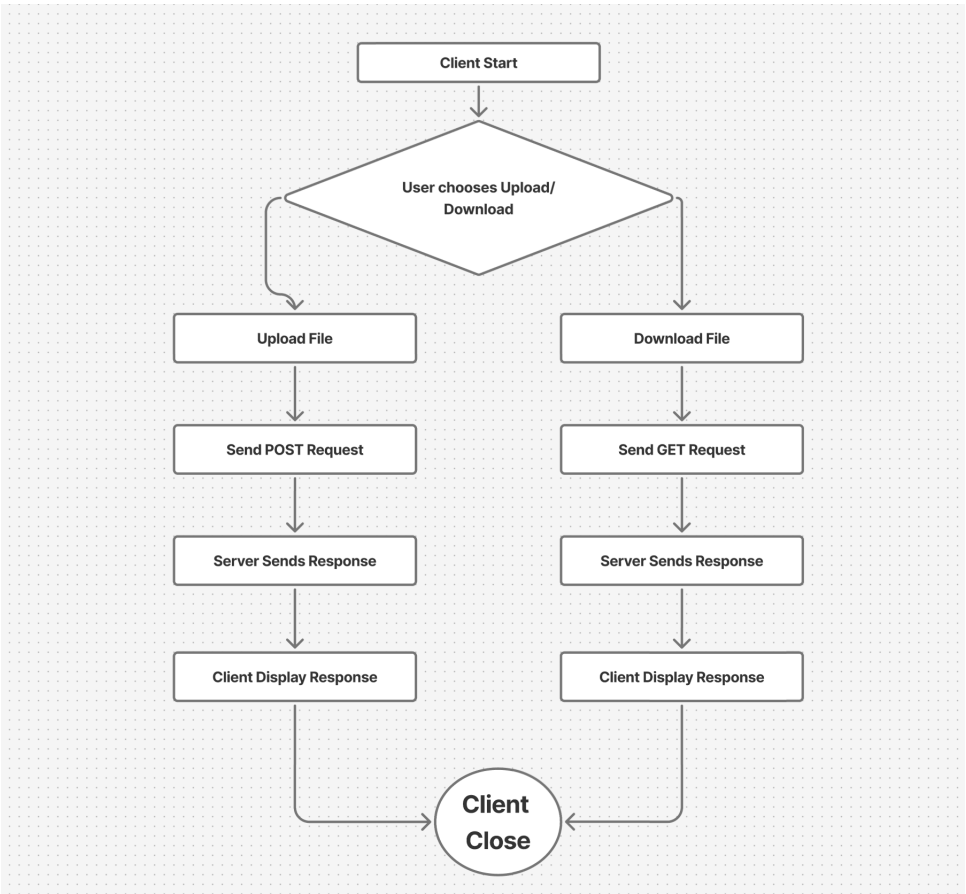


Figure 2: Flowchart of the File Client

## 4.3 Server Code (Screenshots)

```
static void handleUpload(HttpExchange exchange, String uploadDir) throws IOException {
    String method = exchange.getRequestMethod();
    if (!method.equalsIgnoreCase(anotherString:"POST")) {
        System.out.println(x:"[SERVER] 405 Method Not Allowed for /upload");
        exchange.sendResponseHeaders(rCode:405, -1);
        return;
    }
    String filename = exchange.getRequestHeaders().getFirst(key:"X-Filename");
    if (filename == null || filename.isEmpty()) {
        filename = "upload_" + System.currentTimeMillis() + ".dat";
    }
    File file = new File(uploadDir + "upload_" + filename);
    try (InputStream is = exchange.getRequestBody();
         FileOutputStream fos = new FileOutputStream(file)) {
        byte[] buffer = new byte[1024];
        int bytesRead;
        while ((bytesRead = is.read(buffer)) != -1) {
            fos.write(buffer, off:0, bytesRead);
        }
    }
    String response = "File uploaded successfully as: " + filename;
    System.out.println("[SERVER] 200 OK: Uploaded " + filename);
    exchange.sendResponseHeaders(rCode:200, response.length());
    exchange.getResponseBody().write(response.getBytes());
    exchange.close();
}
```

Figure 3: Server code snippet: File upload handling

```java
static void handleDownload(HttpExchange exchange, String uploadDir) throws IOException {
    String method = exchange.getRequestMethod();
    String query = exchange.getRequestURI().getQuery();
    String filename = null;
    if (query != null && query.startsWith(prefix:"filename=")) {
        filename = query.substring("filename=".length());
    }

    if (!method.equalsIgnoreCase(anotherString:"GET")) {
        String msg = "Method Not Allowed";
        System.out.println(x:"[SERVER] 405 Method Not Allowed for /download");
        exchange.sendResponseHeaders(rCode:405, msg.length());
        exchange.getResponseBody().write(msg.getBytes());
        exchange.close();
        return;
    }
    File file = new File(uploadDir + filename);
    if (!file.exists()) {
        String msg = "File Not Found";
        System.out.println("[SERVER] 404 Not Found: " + filename);
        exchange.sendResponseHeaders(rCode:404, msg.length());
        exchange.getResponseBody().write(msg.getBytes());
        exchange.close();
        return;
    }
    System.out.println("[SERVER] 200 OK: Downloading " + filename);
    exchange.getResponseHeaders().set(key:"Content-Type", value:"application/octet-stream");
    exchange.getResponseHeaders().set(key:"Content-Disposition", "attachment; filename=\"" + filename + "\"");
    exchange.sendResponseHeaders(rCode:200, file.length());
    try (OutputStream os = exchange.getResponseBody();
         FileInputStream fis = new FileInputStream(file)) {
        byte[] buffer = new byte[1024];
        int bytesRead;
        while ((bytesRead = fis.read(buffer)) != -1) {
            os.write(buffer, off:0, bytesRead);
        }
    }
    exchange.close();
}
```

Figure 4: Server code snippet: File download handling

## 4.4 Client Code (Screenshots)

```
static void uploadFile(String filePath) {
    try {
        File file = new File(filePath);
        if (!file.exists()) {
            System.out.println(x:"File does not exist.");
            return;
        }

        URI uri = URI.create(baseURL + "/upload");
        URL url = uri.toURL();

        HttpURLConnection connection = (HttpURLConnection) url.openConnection();
        connection.setRequestMethod(method:"POST");
        connection.setDoOutput(dooutput:true);
        connection.setRequestProperty(key:"X-Filename", file.getName());

        try (FileInputStream fileInputStream = new FileInputStream(file);
             OutputStream outputStream = connection.getOutputStream()) {
            byte[] buffer = new byte[1024];
            int bytesRead;
            while ((bytesRead = fileInputStream.read(buffer)) != -1) {
                outputStream.write(buffer, off:0, bytesRead);
            }
        }

        int responseCode = connection.getResponseCode();
        String responseMessage = connection.getResponseMessage();
        System.out.println(responseCode + " " + responseMessage);

        InputStream responseStream = (responseCode >= 200 && responseCode < 400)
                ? connection.getInputStream()
                : connection.getErrorStream();

        if (responseStream != null) {
            BufferedReader reader = new BufferedReader(new InputStreamReader(responseStream));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println("Server Response: " + line);
            }
            reader.close();
        }

        connection.disconnect();

    } catch (Exception e) {
        System.out.println("Error uploading file: " + e.getMessage());
    }
}
```

Figure 5: Client code snippet: Upload request

```java
static void downloadFile(String filename) {
    try {
        URI uri = URI.create(baseURL + "/download?filename=" + filename);
        URL url = uri.toURL();

        HttpURLConnection connection = (HttpURLConnection) url.openConnection();
         connection.setRequestMethod(method:"GET");
        // connection.setRequestMethod("POST");

        int responseCode = connection.getResponseCode();
        String responseMessage = connection.getResponseMessage();

        if (responseCode == HttpURLConnection.HTTP_OK) {
            File downloadDir = new File(pathname:"download");
            if (!downloadDir.exists()) {
                downloadDir.mkdirs();
            }
            String outputFile = "download/" + filename;
            InputStream inputStream = connection.getInputStream();
            FileOutputStream fileOutputStream = new FileOutputStream(outputFile);

            byte[] buffer = new byte[1024];
            int bytesRead;
            while ((bytesRead = inputStream.read(buffer)) != -1) {
                fileOutputStream.write(buffer, off:0, bytesRead);
            }

            fileOutputStream.close();
            inputStream.close();

            System.out.println(responseCode + " " + responseMessage);
            System.out.println("Download complete. Saved as: " + outputFile);

        } else if (responseCode == HttpURLConnection.HTTP_NOT_FOUND) {
            System.out.println(responseCode + " " + responseMessage);
            System.out.println(x:"File not found on server.");
        } else if (responseCode == HttpURLConnection.HTTP_BAD_METHOD) {
            System.out.println(responseCode + " " + responseMessage);
            System.out.println(x:"Invalid request method.");

        } else {
            System.out.println("Server returned unexpected response: " + responseCode + " " + responseMessage);
        }

        connection.disconnect();

    } catch (Exception e) {
        System.out.println("Error downloading file: " + e.getMessage());
    }
}
```

Figure 6: Client code snippet: Download request

# 5 Result Analysis

## 5.1 Upload Test

- Client successfully sent a file using POST.
- Server stored the file in `uploads/`.

```
PS D:\Depression\Computer Networking\Lab\Lab 4 Assignment> javac FileClient.java
PS D:\Depression\Computer Networking\Lab\Lab 4 Assignment> java FileClient
2. Download File
Choose: 1
Enter file path to upload: test.pdf
200 OK
Server Response: File uploaded successfully as: test.pdf
```

Figure 7: Client output during file upload

```
PS D:\Depression\Computer Networking\Lab\Lab 4 Assignment> java FileServer
Server started on port 8080
[SERVER] 200 OK: Uploaded test.pdf
[SERVER] 404 Not Found: e.txt
[SERVER] 404 Not Found: test.pdf
[SERVER] 200 OK: Downloading upload_test.pdf
[SERVER] 405 Method Not Allowed for /download
```

Figure 8: Server console showing received file upload

## 5.2 Download Test

- Client requested file with GET.
- Server returned the file.
- File saved locally with prefix `client_`.

Both operations returned correct HTTP status codes (`200 OK`, `404 Not Found`, etc.).

```
PS D:\Depression\Computer Networking\Lab\Lab 4 Assignment> java FileClient
2. Download File
Choose: 2
Enter filename to download: e.txt
404 Not Found
File not found on server.
```

Figure 9: Client output during file download

```
PS D:\Depression\Computer Networking\Lab\Lab 4 Assignment> java FileClient
1. Upload File
2. Download File
Choose: 2
Enter filename to download: upload_test.pdf
200 OK
Download complete. Saved as: download/upload_test.pdf
```

Figure 10: Server console showing file download request

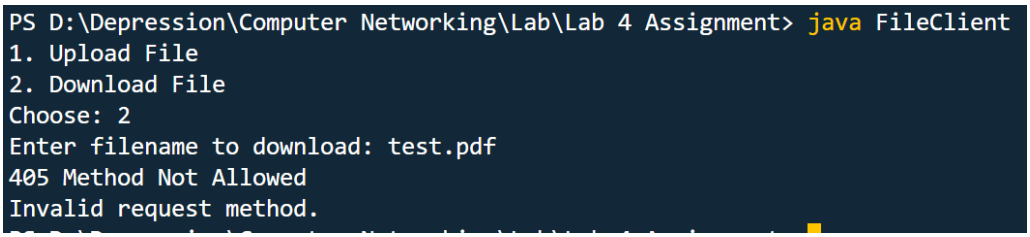


Figure 11: Error response: Method Not Allowed

# 6 Discussion

## 6.1 Socket Programming Approach

- Uses raw TCP connections.
- Requires manual parsing of requests and headers.
- Provides fine-grained control but is complex.

## 6.2 HTTP GET/POST Approach

- Uses Java's built-in `HttpServer` and `HttpURLConnection`.
- Higher-level, easier, and aligns with modern REST APIs.
- More reliable error handling.

## 6.3 Learnings

- Difference between raw sockets and HTTP abstraction.
- Practical implementation of file upload and download.
- Handling multiple request methods in networking applications.

## 6.4 Difficulties Faced

- Handling request streams correctly.
- Ensuring files were saved with proper names and extensions.
- Debugging incorrect HTTP headers.

# 7 Conclusion

We successfully implemented a file transfer system in Java using HTTP GET and POST requests. The experiment demonstrated the simplicity and power of HTTP for file sharing compared to raw sockets, making it more practical for real-world applications.